13 What are OAuth 2 and OpenID Connect?

This chapter covers

- The purpose of access tokens
- How tokens are issued and validated in an OAuth 2 system
- The roles involved in an OAuth 2/OpenID Connect system

Suppose you work for a large organization and use several tools in your daily work. You use bug tracker apps, apps for documenting your work, apps for registering your time, and so on. In each one, you need to authenticate tools to work with them. Would you use different sets of credentials for these apps? Of course, doing so could work, but this approach would be cumbersome for the user (you), and it would also complicate the purpose of the apps you work with.

For you, the complexity comes from the fact that you'd have to remember the credentials and log in several times in each of the apps you use. For the apps, the added complexity comes from the fact that they'd also need to implement the capability of persisting and protecting the credentials and the actual authentication.

What about managing the responsibility for storing the credentials and authentication in a separate app? In this case, the users would only have to log in once and use all their apps without bothering to authenticate repeatedly. Is there such a solution? Yes. You can implement authentication over the OAuth 2 specification. The second thing is that we can go even bigger. An app that can be used by public users (users outside of an organization—an app you create for the whole world) might also need authentication capabilities. The app can implement those capabilities, but

- Implementing authentication in this app requires more work and effort.
- The users need a specific set of credentials for this app.
- The users sometimes don't trust creating specific credentials for any small app they use.

Could you allow the users of this app to log in with some credentials they already have? Could your app's users log in with their Facebook, GitHub, Twitter, or Google account credentials instead, maybe? You might have already seen that all over the place. Apps around the web allow users to choose to sign up and use various social media platforms. This way, an app allows its users to authenticate with a set of credentials they already have without you needing to implement something specific in your app. Such an approach

- Reduces costs (e.g., of implementing and maintaining authentication in your app)
- Avoids user trust issues (e.g., of having to register and create another set of credentials that your app will maintain)
- Helps users minimize the number of credentials they use

OAuth 2 is a specification that tells one how to separate the authentication responsibilities in a system. This way, multiple apps can use one other app that implements the authentication, helping the users to authenticate faster, keeping their details more secure, and minimizing the costs of implementation in the apps.

We'll start with section 13.1, where I'll introduce the main actors involved in a system where the authentication and authorization are built over the OAuth 2 specification. In section 13.1, you'll learn all the responsibilities of the parts of an OAuth 2 system, such as the user, client, authorization server, and resource server. In section 13.2, we discuss tokens. Tokens are like access keys for an app. You'll learn you can use multiple types of tokens and when it's best to use each type. Section 13.3 reviews the most critical ways tokens can be issued (which we'll implement and test in chapter 14). We end this chapter with section 13.4, where we'll go through possible pitfalls you need to consider when implementing OAuth 2.

Before we start, I'd like to mention that in this chapter, I offer a simple perspective of everything you need to know to further understand our discussion in chapters 14 through 16. I don't intend to make you an OAuth 2 and OpenID Connect expert in one chapter. I don't think that'd be possible, as the two are quite complex enough for others to have written entire books about them. If you wish to expand your knowledge on the subject, I recommend *OAuth 2 in Action* by Justin Richer and Antonio Sanso (Manning, 2017) and *OpenID Connect in Action* by Prabath Siriwardena (Manning, 2023).

13.1 The big picture of OAuth 2 and OpenID Connect

Suppose you need to attend an interview with a large corporation. You have been called to their main headquarters for a face-to-face discussion. But not just anyone can enter the company's office. There are specific protocols implemented for visitors.

To get in the building and attend the discussion, you must first visit the front desk and prove who you are by using an ID. After getting identified, you'll get an access card from the front desk, allowing you to open certain doors. You might not even be able to use all the elevators but only specific ones (figure 13.1).



Figure 13.1 The OAuth 2 specification is very similar to accessing an office building.

The process of getting into the building for the discussion is very similar to how authentication and authorization work in an OAuth 2 implementation. You are the user who needs to execute a specific use case (go to a particular room for the discussion). For that, you use your credentials (the ID) to authenticate at the front desk (the authorization server). Once you prove who you are, you have an access card (the token). But you can use this token only to access specific resources (like elevators and specific doors). You can use the access card only for a short period. After your discussion, you must return the card to the front desk.

In this section, we discuss the responsibilities that interact with one another in an OAuth 2 system, and you'll find out how is it similar to visiting an organization's headquarters for an interview. We also discuss what OAuth 2 is as a specification and the difference between OpenID Connect (a protocol) and OAuth 2 (the specification it relies on). I consider it essential to understand well the concepts behind the scenes of this authentication and authorization approach before diving into its implementation in chapters 14 through 16.

First, let's discover who plays roles in an OAuth 2 system. Figure 13.2 presents the main actors of an OAuth 2 system. By actor, I mean any entity that plays a role in the system's functionality. With an OAuth 2 system, you'll find the following actors:

- *The user*—The person who uses the application. The users usually work with a frontend application, which we call a client. Users don't always exist in an OAuth 2 system, as discussed in section 13.3.3, where you'll learn about the client credentials grant type.
- *The client*—The application that calls a backend and needs authentication and authorization. The client can be a web app, a mobile app, or even a desktop app or a separate backend service. The system usually doesn't have a user when the client is a backend service.
- *The resource server*—A backend application that authorizes and serves calls sent by one or more client applications.
- *The authorization server*—An app that implements authentication and safe storage of credentials.

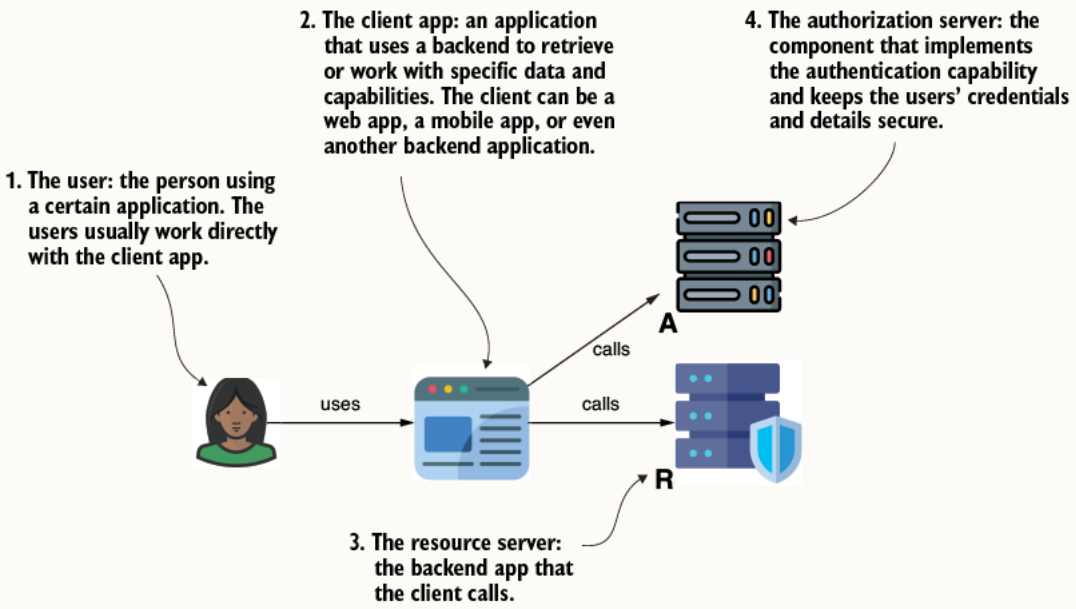


Figure 13.2 The participants in an OAuth 2 framework. Users interact through a client that requires authorization for certain operations on the backend service, known as a resource server. For backend authorization, the client's initial step is authentication by the authorization server.

Let's now discuss how authentication and authorization really happen. The steps are simple:

1. The user executes a certain use case with the client app.
2. The client app gets authorized to call the resource server to serve the user's request.
3. To get authorized, the client first requests a token (called an access token) from the authorization server. This token is just some specific information that helps the client to prove the authorization server identified them correctly.
4. The client uses the token the authorization server issues to get authorized when sending requests to its backend (the resource server).

Figure 13.3 details a bit of the flow visually. The numbered steps in the figure represent the following:

1. The user tries to use the client application to execute a particular use case.
2. The client application knows it can't call its backend without first having a token that will allow it to get authorized. The client requests such an access token from the authorization server.
3. Following the client app's request, the authorization server issues a token and sends it to the client app.
4. The client uses the token to send requests to its backend (the resource server).
5. The resource server authorizes the client's request. If authorized successfully, the resource server executes the client's request and replies back.
6. The client shows the result to the user.

Figure 13.3 The simplest explanation of the OAuth 2 authentication procedure involves the client obtaining a token from the authorization server. This token is then used to gain authorization for requests sent to the backend application, or resource server.

But what exactly is this token that the authorization server issues? A token can be any piece of data (usually a string of characters) that allows the client to prove they (and/or the user) have been identified by the authorization server. The token is also a way to get more details about both user and client if needed. Because the authorization server now manages all the user and client details, the backend sometimes needs to get part of these details from the authorization server and use them. The backend will get such details by means of the token. Sometimes, the token itself contains the needed details (as you'll read in section 13.2, such tokens are called non-opaque tokens); otherwise, the backend needs to call the authorization server to get the data about the client and the user (i.e., opaque tokens). In addition, unlike a physical key, an access token doesn't have a large lifespan. It expires after a short period (in most cases minutes), after which the client needs to ask the authorization server again for another token. This way, a lost token (such as a lost key) can't be misused.

OAuth 2 describes multiple flows in which a client might get a token. We call these flows "grant types," and in section 13.3, we discuss the most common grant types used.

13.2 Using various token implementations

Tokens are the access cards (figure 13.4) a client uses to get authorized when sending requests to the backend (the resource server). Tokens are an essential part of the OAuth 2 authentication and authorization process because they're the ones used to prove the au-

thenticity of a client and user authentication, but they are also the way a backend gets more details about the client and the user.

In this section, we discuss how tokens are classified and, depending on the token type, how they are used in the authorization process.

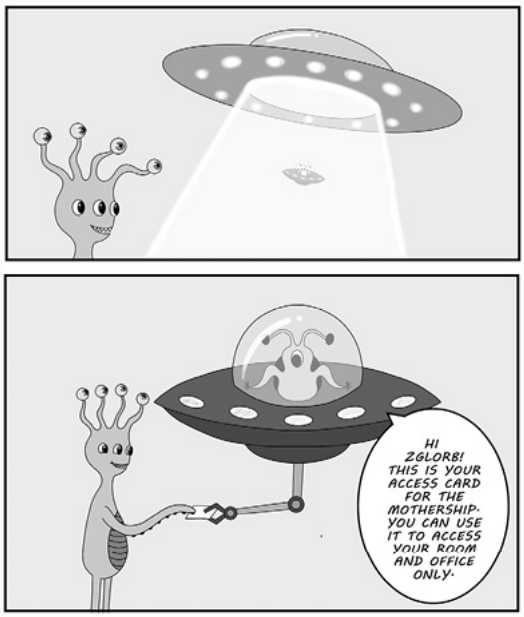


Figure 13.4 Zglorb (the user) needs to access the Mothership (resource server). To do that, they're first authenticated (by the authorization server), and then they're given an access card (the token). Zglorb can only access specific areas (resources) of the Mothership using their access card.

We classify tokens based on the way they provide the resource server with the data for authorization:

- *Opaque*—Tokens which don't store data. To implement the authorization, the resource server usually needs to call the authorization server, provide the opaque token, and get the details. This call is known as the introspection call.
- *Non-opaque*—Tokens that store data, making it immediately possible for the backend to implement the authorization. The JSON Web Token (JWT) is the most used non-opaque token implementation.

13.2.1 Using opaque tokens

Opaque tokens don't contain data the backend can use to identify either the user or the client, or to implement the authorization rules. Opaque tokens are just proof of an authentication attempt. When a resource server receives an opaque token, it needs to call the authorization server to find out whether the token is valid and to get more information to allow it to apply authorization constraints.

An opaque token is literally like a key to a treasure chest. It doesn't provide any information up front; you know it works only when you try opening the chest with it. Once you find out it's valid, it also gets you to what's inside the chest (in this case, the user and client details). Figure 13.5 illustrates this analogy.

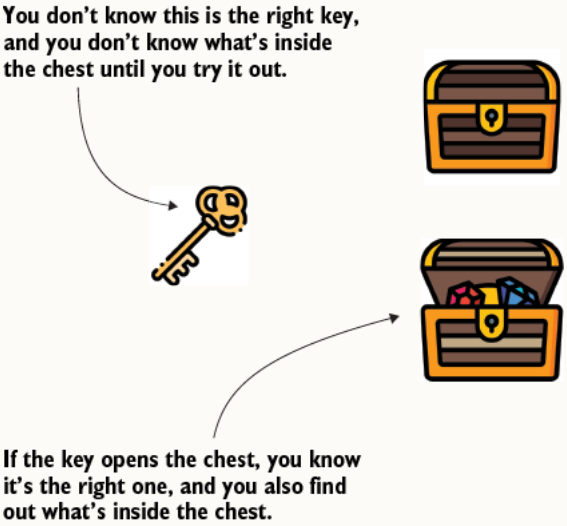


Figure 13.5 An analogy to opaque tokens. An opaque token is just like a key. You don't know if it works until you try it. If the key works, you also get access to what's inside it.

The resource server calls an endpoint provided by the authorization server to find out whether the opaque token is valid and obtain the needed details about the client and the user to whom the token was issued. This call is named a token introspection (figure 13.6). Once

the resource server has these details, it can apply the authorization constraints.

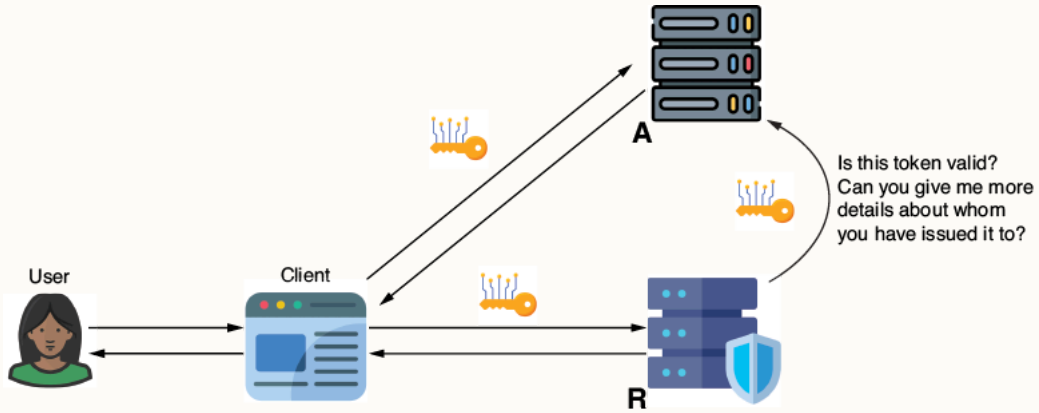


Figure 13.6 The token introspection call. The resource server sends a request to the authorization server to find out whether the opaque token is valid and details about to whom it was issued.

13.2.2 Using non-opaque tokens

Unlike the opaque tokens we discussed in section 13.2.1, non-opaque tokens contain information about the client and the user to whom the authorization server issued the tokens during the authentication process. You can compare non-opaque tokens with signed documents (figure 13.7).

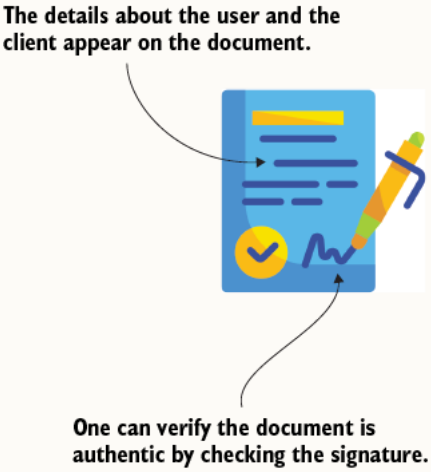


Figure 13.7 The non-opaque token is like a signed document. It contains the needed details for the resource server to apply the authorization constraints and a signature to validate its authenticity.

The most common implementation of a non-opaque token is the JWT. A JWT is composed of three parts (figure 13.8):

- *The header*—Usually contains data about the token, such as the cryptographic algorithm used to sign the token or the key ID that the authorization server used to sign it
- *The body*—Usually contains data about the entity to whom the token was issued, such as the client and the user details
- *The signature*—A cryptographically generated value that can be used to prove that the authorization server indeed issued the token and that no one altered its content (in the header or body) after it was generated

The data in the header and body is JavaScript Object Notation (JSON) formatted, then encoded as Base64 to make it smaller and easier to transfer. Dots separate the three parts.

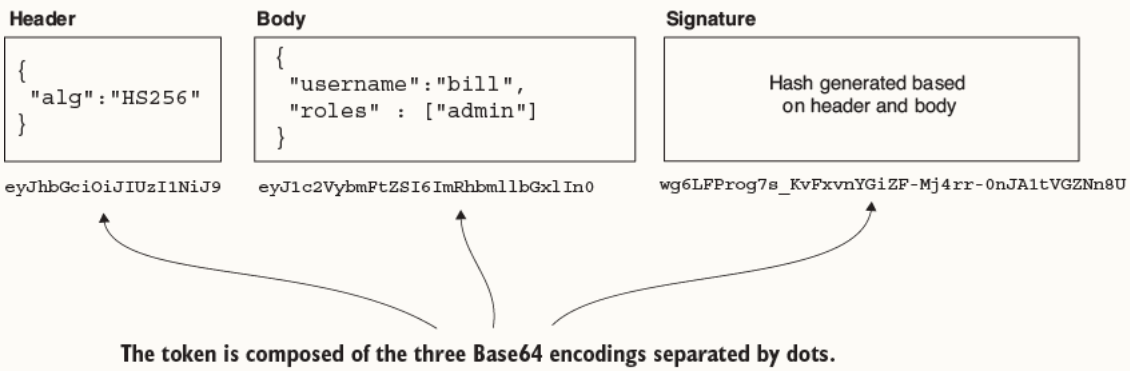


Figure 13.8 The anatomy of a JWT token. The header and the body contain the details needed for the resource server to validate the token's authenticity and apply the authorization constraints.

The next snippet shows an example of a JWT where the three parts are Base64 encoded, separated by dots:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4f
```

You may be asking yourself now, "When should I use opaque tokens, and when should I use non-opaque ones?" As I told you a bit earlier in this section, non-opaque tokens are the most frequently used today because they don't require introspection to validate them. However, non-opaque tokens contain data, and the client sends this data over the wire to its backend. Anyone obtaining the token can also see the data the token carries. In the majority of cases, this is not a problem. And I recommend anyone avoid sending too much data within the token.

But what should you do if you have a larger amount of data or data that is unsafe to send over the wire carried within a token? In such a case, opaque tokens might be a good alternative. I recommend you first consider non-opaque tokens and fall back to opaque ones only if the amount of data to be carried by the token is too large, or you need to send details that are more sensitive, and you want to avoid exchanging them through the token.

13.3 Obtaining tokens through various grant types

This section discusses grant types. A grant type is a process in which a client gets a token. In apps, you'll find various approaches in which a client gets a token from the authorization server. We'll discuss the three most used grant types. At the end of this section, we'll explore how a client can re-generate a token after it expires.

NOTE You might still find apps implementing two other grant types: the implicit and the password grant type. These two grant types became deprecated because they were found to not be secure enough. We won't discuss them in this book; I don't recommend using them in apps. You can always replace either with one of the other grant types discussed in this section. If you want to learn more about the password grant type, there is a good discussion about it in chapter 12 of the first edition of this book. We also go quickly over the implicit grant type and why it's deprecated when discussing the authorization code grant type in this section.

Section 13.3.1 discusses the authorization code grant type, which is the most used grant type when the system needs to allow a user to authenticate. In section 13.3.2, we discuss an addition to the authorization code grant type—the proof key for code exchange (PKCE). In section 13.3.3, we continue with the situation where an app needs to get a token without having a user authenticate, and we end with how to re-generate tokens in section 13.3.4.

13.3.1 Getting a token using the authorization code grant type

The authorization code grant type is the most used grant type today. It is used when our app needs to authenticate a user (to easily understand this grant type, see figure 13.9, which illustrates the steps in a sequence diagram):

1. The user wants to do something within the app they use. For example, let's say that the girl on the left side of the diagram is Mary, an accountant who wants to see all the invoices that the company she works for needs to pay.
2. The app Mary uses is the client. In this case, Mary stays in front of her computer, so her client app is a web app. But Mary could have also used the mobile version of the app. In both cases, the grant type would have looked the same. Because Mary didn't log in, the app redirects her to a login page hosted by the authorization server.
3. Now Mary sees the login page in her browser. The login page is not in the app she accessed but is hosted by a different system. Mary recognizes the page she was redirected to as the central authentication application that she uses for any of the apps she works with for the company. Mary knows that after submitting her credentials, the browser will return her to the invoices application, and she will be able to see the invoices and operate with the data she needs. She fills in the correct credentials and selects the login button.

4. Because the credentials Mary provided are correct, the authorization server redirects back to the invoices application. The authorization server also provides the initial application (the client) a unique code named an "authorization code." The client will use this code to get an access token.
5. The client asks for an access token. The client needs this access token to send requests to its backend (the resource server).
6. Because the authorization code is correct (the same one the server provided in step 5), the authorization server responds with an access token.
7. The client app uses the access token to send a request to its backend and get authorized.

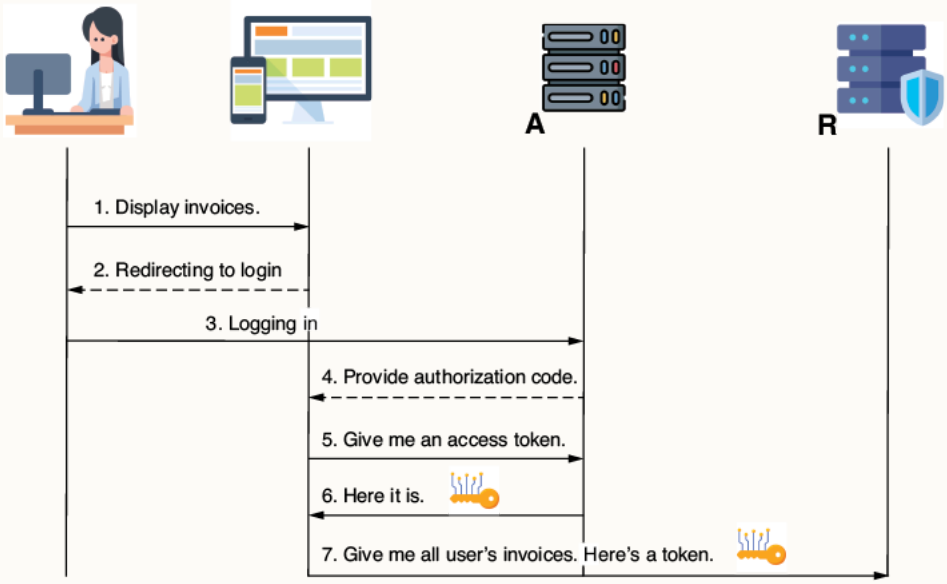


Figure 13.9 The authorization code grant type. The user is required to sign in. After login, the authorization server issues an authorization code to the client, which the client then utilizes to obtain an access token. This access token enables the client to have its requests authorized by the resource server.

A few observations to help you understand this flow better:

- Pay attention to the dotted arrows. It's essential to remember that those represent redirects in the browser and not requests or responses. At step 2, the client app redirects the user to the authorization server login page (it redirects in the browser to a web page of another app). At step 4, the authorization server redirects back to the client app providing the authorization code (usually as a query parameter).
- Mary (the user) isn't aware of steps 4 through 7. After she logs in, she will, in the end, see the invoices displayed by the client who will get them in response to step 7 after being authorized.
- Remember not to confuse the authorization code and the access token. The access token is what the client needs, in the end, to get authorized by its backend (step 7). But to get the access token, the client first gets an authorization code (steps 4 and 5).

In addition, many developers new to authorization and authentication are confused about step 4. The question I frequently get is, "Why doesn't the authorization server directly return the access token here?" It looks strange that the client needs to take one more step to get the access token when they could have directly gotten it at step 4.

But it makes sense. In fact, in the first version of OAuth, the authorization server provided the access token instead of the authorization code at step 4. This is what we now name the "implicit grant type," which is deprecated and no longer recommended to be used. The reason is that a redirect can be easily intercepted, and a bad-intentioned individual could have very easily obtained the access token. By returning the authorization code, the authorization server forces the client to re-send a request where they must authenticate again with their credentials. This way, if someone intercepts the redirect and gets the authorization code, it's not enough to get an access token. They would need to also know the client credentials to send the request and get the token.

Figure 13.10 visually presents the two steps where the authorization code adds supplementary protection to avoid allowing someone to get the access token.

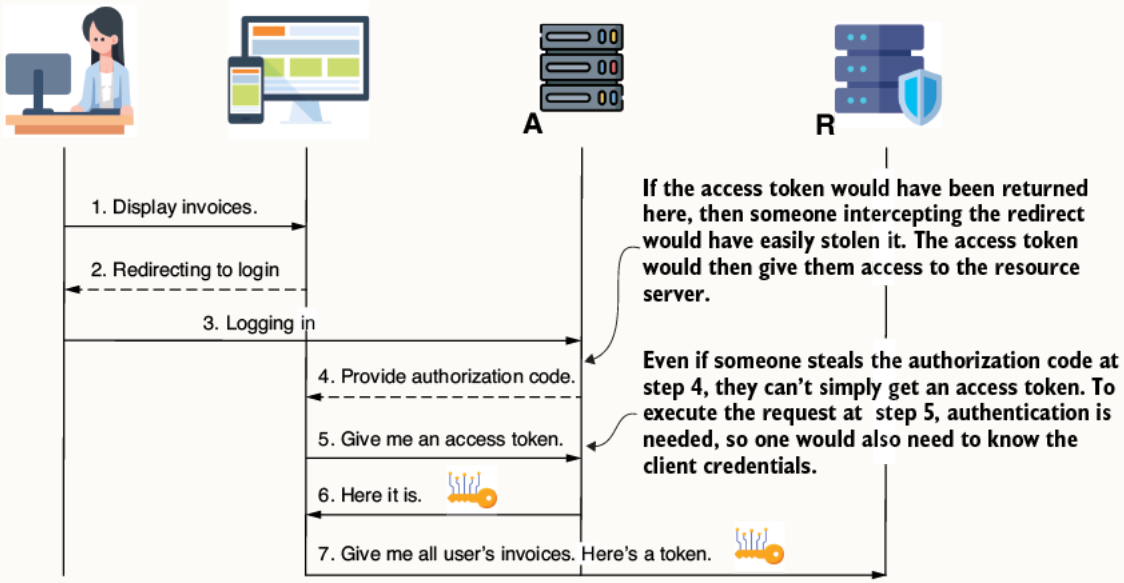


Figure 13.10 After logging in and receiving an authorization code, the client must make an additional request to acquire the access token. During this request, the client is required to verify its identity using its credentials. This method increases the challenge for anyone attempting to illicitly obtain the access token, as they would need to intercept the authorization code and also know the client's credentials.

13.3.2 Applying PKCE protection to the authorization code grant type

What if a bad-intentioned individual manages to get the client credentials as well? In this case, they could obtain an access token and send requests to the resource server. Is there a way in which we can prevent something like this from happening? Yes, the proof key for code exchange (PKCE, usually pronounced "pixy") is an enhancement added to the authorization code flow to make it more secure. In this section, we discuss how PKCE covers the case where someone could get an access token by stealing the client credentials.

Using PKCE only affects two of the steps of the authorization code grant type, which we discussed in section 13.3.1. In figure 13.11, I made the arrows representing steps 3 and 5 thicker. These two are the steps of the authorization code grant type where PKCE is applied:

1. First, the client needs to generate a random value. This value can be a random string of bytes. This value is called *the verifier*.
2. Second, the client will apply a hash function over the randomly generated value at step 1. A hash function is encryption characterized by the fact that the output cannot be turned back to the input (chapter 4). The result of applying the hash function over the verifier is called *the challenge*.

```
verifier = random();
challenge = hash(verifier);
```

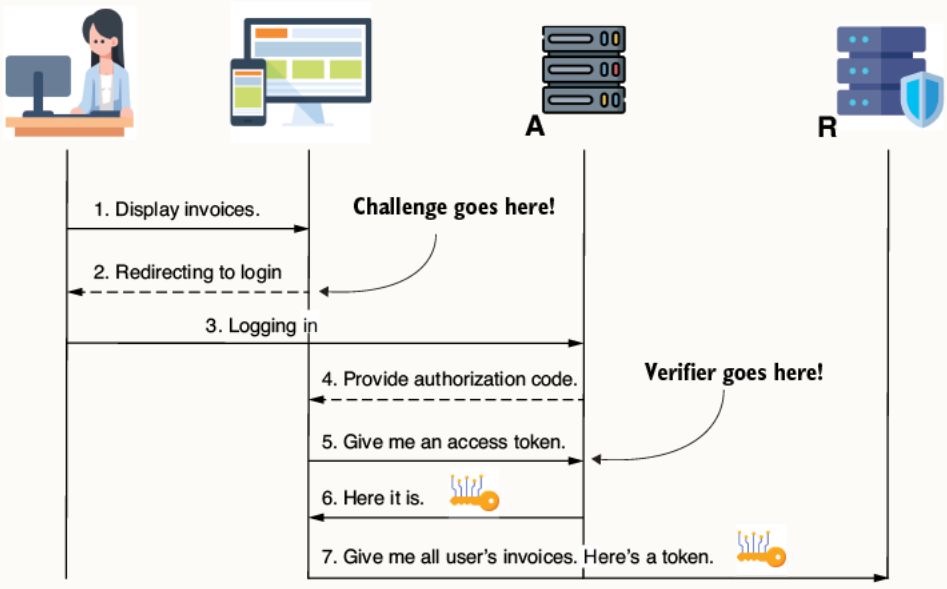


Figure 13.11 The client will send a challenge in step 3 and the verifier in step 5 to prove that they are the same client who initially asked the user to log in.

The client sends the challenge in step 3 with the user login. The authorization server keeps the challenge and expects the verifier with the request made at step 5 to get the access token. If the verifier the client sends when requesting the token at step 5 matches the

challenge they sent at step 3, then the authorization server knows the client app that requests the token is the same one that requests the user to authenticate.

Someone cannot get an access token now even if they somehow manage to get the authorization code at step 4. This is because they would need to know the verifier value as well. They can't know the verifier because the client has not yet sent it on the wire. And they can't get the verifier simply by intercepting the challenge (at step 3) because the challenge is created with a hash function, implying that the output can't be turned back into the input.

13.3.3 Getting a token with the client credentials grant type

Sometimes an app needs to get authorization without user intervention. If there's no user on the scene, an app will have to use the client credentials grant type to get an access token. This situation commonly happens when a service needs to call another service when an objective event, such as the timer of a scheduled process, triggers it. Using the client grant type, the app only needs to authenticate with its client credentials. Figure 13.12 presents the client credentials grant type:

1. The app requests an access token from the authorization server. The app uses their credentials to authenticate.
2. If credentials are valid, the authorization server issues an access token.
3. The app uses the access token to get authorized when sending a request to the resource server.

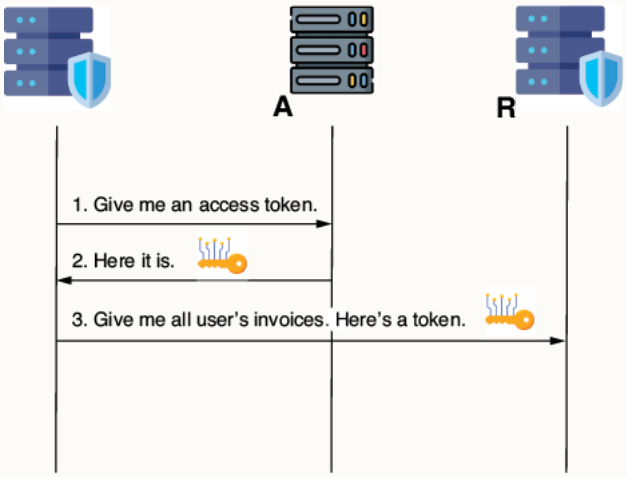


Figure 13.12 The client grant type. An app gets an access token without needing a user to authenticate.

13.3.4 Using refresh tokens to get new access tokens

One essential thing you must remember about tokens is that they must have a relatively short life span. The exact time they stay alive is usually decided according to the scenario, but it's usually as short as 15 minutes, and I have never used tokens that live more than one hour. Eventually, all tokens need to expire sooner or later. When a token expires, the resource server won't accept it anymore. In such a case, when a client has a token, and the token has expired, they have two options:

1. Get a new access token by repeating the grant type steps. This implies asking the user to log in again in case of the authorization code grant type.
2. Use a refresh token to get a new access token.

Refresh tokens are particularly useful when the client uses a grant type, such as the authorization code, which implies a user needs to log in. Imagine you have tokens with a 15-minute lifespan. As a user, would you not be bothered if your app asked you to log in again and again every 15 minutes? I would!

The app can use the refresh token to get a new access token instead of asking the user to log in every time the access token expires.

Figure 13.13 shows the steps for using the refresh token:

1. The user tries to get some data, which implies that the client must call its backend.
2. Because the access token (previously obtained) expired, the client needs to get a new one. The client sends a refresh token to prove they are the same one who had already authenticated before.
3. The authorization server recognizes the refresh token and provides the client with a new access token.
4. The client can call the backend (the resource server) and get authorized with the new access token.

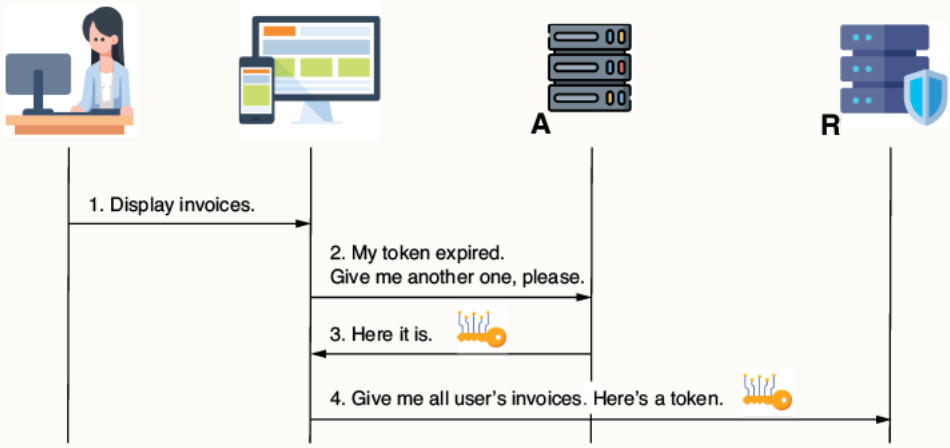


Figure 13.13 A client app can use a refresh token to get a new access token when the old one expires. This way, the app avoids requesting the user to authenticate again.

13.4 What OpenID Connect brings to OAuth 2

There's certainly still a lot of confusion out there about OpenID Connect (sometimes named OIDC) and OAuth 2 and the differences between the two. I usually tell my students to stress less about this subject: "If you understand OAuth 2, you also know how to use OpenID Connect."

In fact, OIDC is a protocol built over the OAuth 2 specification. For this reason, understanding OAuth 2 helps you easily get OIDC. Let me give you an analogy for specifications versus protocols.

We all use electrical sockets every day. Electrical sockets look different around the world. Sometimes, this is a real pain when you travel. You may need to have adapters to ensure you can charge your devices, especially if you travel between different geographical regions.

But behind the scenes, all outlets work the same way. There are some wires that output the electrical voltage. You can define a framework on which all the electrical sockets in the world work in only a couple of bullet points:

- An electrical socket has three wires that allow electrical current to flow: the phase, the null, and a grounding. The grounding is optional.
- The electrical socket provides a voltage that can be either around 120 Volts or 230 Volts.

Now, no worries if you're not a technical person; you don't need to understand these two bullets. At least not for learning Spring Security. Just take my word for it.

The problem is that even if all sockets in the world fulfill these specifications, we still get situations where we need adapters to travel. The reason is that they don't have a common protocol. Adapters are required to adapt the outlet from one protocol to another (for example, North America to Europe).

The same happens with apps and authentication and authorization. If two apps fulfill the OAuth 2 specification, they can still run in situations where they are not fully compatible and need to be adapted because they don't run the same protocol. OpenID Connect is a protocol that restricts a bit the liberty of the OAuth 2 specification, introducing a few changes. The major changes are

- Specific values for the scopes (such as *profile* or *openid*).
- The use of an extra token named ID token, used to store the details about the identity of the user and client to whom the token was issued.
- Usually the term *grant type* is also referred to as *flow* when discussing it in terms of OIDC, while the *authorization server* is commonly called the *identity provider* or *IdP*.

13.5 The sins of OAuth 2

This section discusses possible vulnerabilities of the applications using OAuth 2 authentication and authorization. It's essential to understand what can go wrong when using OAuth 2 to avoid these scenarios when developing applications. Of course, like anything else in software development, OAuth 2 isn't bulletproof. It has its vulnerabilities, which we must be aware of when building our applications. I list here some of the most common:

- *Using cross-site request forgery (CSRF) on the client*—With a user logged in, CSRF is possible if the application doesn't apply any CSRF protection mechanism. We had a great discussion on the CSRF protection implemented by Spring Security in chapter 9.

- *Stealing client credentials*—Storing or transferring the credentials unprotected can create breaches that allow attackers to steal and use them.
- *Replaying tokens*—As discussed in section 13.2, tokens are the "keys" we use within an OAuth 2 authentication and authorization architecture to access the resources. You send them over network, and sometimes they might be intercepted. If intercepted, they are stolen and can be reused. Imagine you lose the key from your home's front door. What could happen? Somebody else could use it to open the door as many times as they like (replay).
- *Token hijacking*—You interfere in the authentication process and steal tokens you can use to access resources. This is also a potential vulnerability of using refresh tokens, as they, too, can be intercepted and used to obtain new access tokens. I recommend this helpful article: **http://mng.bz/am5z**.

Remember, OAuth 2 is a framework. The vulnerabilities are the result of implementing functionality wrongly over it. Using Spring Security already helps us mitigate most of those vulnerabilities in our applications. When implementing an application with Spring Security, like you saw in this chapter, we need to set the configurations, but we rely on the flow as implemented by Spring Security.

For more details on vulnerabilities related to the OAuth 2 framework and how a bad-intentioned individual could exploit them, see part 3 of *OAuth 2 In Action* by Justin Richer and Antonio Sanso (Manning, 2017), available at **http://mng.bz/g7Ql**.

Summary

- The OAuth 2 framework describes secure ways in which a backend can authenticate its clients. OpenID Connect is a protocol that implements the OAuth 2 client by applying some constraints for the possible implementations.
- The four main actors in an OAuth 2 system are
  - *The user*—A person who wants to execute a use case
  - *The client*—An app that must be authorized to access a resource or use case on a given backend
  - *The resource server*—A backend that needs to authorize a client to execute a specific use case or access a resource
  - *The authorization server*—An app that manages the user and client details, allowing them to authenticate, and that provides a token that can be used for means of authorization
- The token is an access card (or a key) a client gets from the authorization server and uses to get authorized to call a use case or access a specific resource on a secured backend (the resource server).
- We classify tokens into two categories:
  - *Opaque*—tokens that don't contain details about the user and client for whom they were issued. For such tokens, a resource server always needs to call the authorization server to validate the token and get the details it needs to authorize the request. This request for the token validation is called introspection.
  - *Non-opaque*—tokens that contain details about the user and the client to whom they've been issued. The most common implementation of non-opaque tokens is the JSON Web Token (JWT).
- There are multiple flows in which a client app can ask the authorization server for a token. These flows in which the token is issued are called grant types. The most common grant types are
  - The authorization code grant type
  - The client credentials grant type
- Sometimes we add supplementary security to the authorization code grant type using a proof key for code exchange (PKCE) approach. Here the client uses additional values to avoid someone being able to obtain access tokens by stealing the client credentials and the authorization code.
- In specific cases, an app might need to get new access tokens without having the user re-authenticate. For such cases, the app can use refresh tokens. Refresh tokens are special tokens that can only be used to get new access tokens.

Previous chapter
‹ Part 4. Implementing OAuth 2 and OpenID Connect

Next chapter
14 Implementing an OAuth 2 authorization server ›